UNIVERSITY OF
CAMBRIDGE
Department of Computer
Science and Technology

Accelerating Molecular Generation through Representation Alignment

Shreyas Ravishankar

Hughes Hall

June 2026

Submitted in partial fulfillment of the requirements for the
Master of Philosophy in Advanced Computer Science

Total page count: 39

Main chapters (excluding front-matter, references and appendix): 28 pages (pp 7–34)

Main chapters word count: 8895

Methodology used to generate that word count:

```
$ make wordcount
gs -q -dSAFER -sDEVICE=txtwrite -o - \
   -dFirstPage=7 -dLastPage=36 report-draft.pdf | \
egrep '[A-Za-z]{3}' | wc -w
8895
```

Declaration

I, Shreyas Ravishankar of Hughes Hall, being a candidate for the Master of Philosophy in Advanced Computer Science, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. [The project required the approved use of {insert name of technology} and such use is acknowledged in the text at the relevant sections.] [I am content for my report to be made available to the students and staff of the University.]

Signed:

Date:

Abstract

Write a summary of the whole thing. Make sure it fits on one page.

Contents

Chapter 1

Introduction

The evolution of generative machine learning in scientific domains is defined by the shifting tensions between data scale, data quality, and structural priors. Our report examines a recent addition to this story, *representation alignment*, in two regimes for de novo molecular generation — small molecules and protein backbones — and characterises its behaviour. In this introduction, we trace the narrative arc of research trends in molecular generation, and highlight the motivation for our work: the emerging need for data-efficient modelling in a world of prior-light models.

1.1 A history of trade-offs

The dominant ideas in generative modelling are *denoising diffusion* and its offshoot *flow matching* [1, 2], which found breakthrough success in the realm of image generation. This paradigm has since been adapted for de novo design challenges across scientific fields, such as generating protein backbones [18, 19, 14], inorganic materials [21], and drug-like small molecules [15]. Training these models is typically expensive in both data and compute, with state-of-the-art image generators routinely training on billions of examples [29]. However, the corpus of available high-quality molecular data sits orders of magnitude below this. Experimental determination is slow and expensive, which means datasets are correspondingly small. For example, protein structure generators have largely been trained on at most half a million structures [14], which may explain why they have not yet matched the maturity of their vision-domain counterparts.

Historically, researchers compensated for the small scale of high-quality molecular data by encoding structural priors directly into model architectures [35]. SE(3)-equivariant architectures, for example, enforce the rotational and translational symmetries expected of molecules by construction [37]. But hard structural priors come with a computational cost, and the operations required to encode geometry are a bottleneck to scaling models.

A recent wave of research has moved entirely the other way: drop hard priors, use bare-bones non-equivariant transformer stacks, and rely on data scale and augmentation to teach the model what equivariance buys. This shift fits a broader trend in machine

learning on leveraging representation instead of model architecture to inject soft inductive biases for structured data. Vision Transformers [38] showed that the spatial-locality and translation-equivariance priors of convolutional neural nets (CNNs) can be learned rather than imposed, given sufficient scale. Graph Transformers like TokenGT [39] have made the same case for graphs. AlphaFold 3 [41] stripped many of the SE(3)-equivariant biases of AlphaFold 2 [40], replacing the structure module with a diffusion model operating on raw coordinates. Brehmer et al. [42] make the empirical scaling case directly: more data through a simpler model beats less data through a richer one. In our work, we examine Tabasco [15] and Proteina [14], which are flow-based generators built on largely vanilla transformers for small molecules and protein backbones respectively.

Given the difficulty of procuring data in scientific domains, data scale must necessarily come from non-experimental sources, which may be of lower quality. For example, recent efforts in protein generation have leaned on synthetic structures from folding-model predictions [14, 31] to expand training datasets into the tens of millions. However, even these enhanced corpora are arguably insufficient. Metagenomic databases catalogue over two billion natural protein sequences [33], and the theoretical sequence space at a median protein length [34] is hundreds of orders of magnitude greater still. Even at the frontier of the field's data scale, we sample but a sliver of the distribution we wish to model.

In summary, the field has traded hard structural priors for architectural simplicity, and curated data for synthetic scale. However, this trade-off dilutes the structural signal available to a model. Furthermore, even our best attempts at creating large datasets may not be large enough. These twin challenges amplify the need for techniques that make training still more efficient without sacrificing quality.

1.2 Representation alignment

But why is training diffusion-transformer architectures expensive in the first place? One answer comes from Yu et al. [5], who argue that these models suffer from a *representation bottleneck*. During training, a model is implicitly asked to learn two distinct tasks simultaneously: (1) representation: extracting semantically meaningful encodings from complex data; and (2) generation: constructing data from these encodings. The standard denoising objective alone may not provide sufficient signal for representation learning, leading to slow convergence and the large training budgets these models require in practice. In the context of molecular generation, this representation burden is compounded by the twin challenges outlined above. Models are now forced to encode complex structural geometry from datasets that only sparsely sample the true underlying distribution, all while operating without the aid of architectural priors.

Yu et al. propose Representation Alignment (REPA) as a remedy. The technique adds a regularisation term that aligns a generator's intermediate hidden states with the embeddings of a frozen pretrained encoder. On image diffusion, they report roughly a 17.5×

training speed-up against a strong baseline to achieve similar generation quality. REPA has since been extended beyond images to video [9], audio [10], and recently to molecular generation [7, 8], where it has been demonstrated but not systematically characterised.

1.3 Contributions

In this light, the core question we investigate in this report is: *can prior-light molecular generation models benefit from representation alignment with pre-trained models, in training efficiency and generation quality?* We aim to characterise when and why this works, including failure modes and limitations.

We document two lines of enquiry.

(i) *Does REPA work in the molecular domain?*: We conduct empirical studies on training flow-based generators with REPA in two regimes: Tabasco [15] for small molecules, and Proteina [14] for protein backbones. We find that REPA does not measurably improve Tabasco, but it improves training efficiency and advances the generation quality envelope for Proteina.

(ii) *When does REPA work?*: Following Singh et al. [6] in the realm of image generation, we develop a profiling framework to characterise the suitability of an encoder as a representation target. We measure three properties of an encoder: (1) an upper bound on the structural information it contains (linear-probe recoverability against domain structural labels); (2) a lower bound on what a projector can transfer from that information (projector-based recoverability); (3) the geometric conditioning of its embeddings for the alignment objective (effective rank).

We do not propose a new architecture or alignment loss, and our two studies are not a controlled experiment over the conditions that govern REPA's effectiveness. Instead, they are best read as paired case studies in regimes where those conditions differ.

The remainder of this report is organised as follows. Chapter 2 sets out the background and context needed to read the rest: flow matching, REPA, molecular generation. Chapter 3 discusses what it means for a generative model to be "good" in our domain, and the tensions that question hides. Chapter 4 develops the encoder-profiling framework and uses it to predict where REPA may and may not help. Chapters 5 and 6 present the two paired studies, small molecules first and protein backbones second. Finally, we conclude in Chapter 7 with a discussion of whether what we learn generalises beyond molecular generation.

Chapter 2

Background and Related Work

In this chapter, we introduce the machinery the rest of this thesis depends on. We outline flow matching (§2.1), the generative paradigm underlying the models we study, and representation alignment (§2.2), the regulariser that is our object of study. We then turn to the molecular setting. We situate generative models within the broader de novo molecular design pipeline (§2.3), before presenting the two molecular domains we work in, small molecules and protein backbones, and the flow-matching models we extend, Tabasco and Proteina (§2.4). The treatment is cursory by design. We intend only to guide the reader through this report, and we provide citations for omitted details.

2.1 Flow matching

At first glance, the term *generative modelling* implies the problem of *creating* new artefacts from scratch. However, it is perhaps more accurately described as the problem of *sampling* new artefacts from a complex data distribution that can only be accessed through a finite training set. A common strategy here is to fix a simple distribution we can sample from cheaply — typically standard Gaussian noise $p_0 = \mathcal{N}(0, I)$ — and learn a mapping from it to the desired data distribution p_1.

Flow matching (FM) [2] realises this mapping as a continuous-time transport. The model designer defines a *probability path* between a noise sample and a data sample — a continuous interpolation between them — and the model learns to carry the noise sample along this path. Concretely, it parameterises a time-dependent *velocity field* $v_\theta(x, t)$ that gives the instantaneous direction of motion at each location x and time $t \in [0, 1]$. At inference, integrating the learned ordinary differential equation (ODE) $\dot{x}_t = v_\theta(x_t, t)$ from $t = 0$ to $t = 1$ pushes a fresh noise sample forward into a fresh data sample.

A common choice of probability path is the linear interpolant, $x_t = (1 - t)\, x_0 + t\, x_1$ for $x_0 \sim p_0$ and $x_1 \sim p_1$. The model's target is the *marginal* velocity field at (x, t): the average direction of motion over all noise-data pairs whose paths pass through that point. However, supervising this marginal directly is intractable, since the contributing pairs are not known in advance. Flow matching sidesteps this difficulty with a key observation [2]:

although the marginal field may be intractable, the *conditional* velocity along the path of any specific noise-data pair is relatively trivial. Indeed, for the linear interpolant, it is simply $\dot{x}_t = x_1 - x_0$. Regressing the model on these conditional velocities, averaged over many sampled pairs, recovers the marginal field in expectation, and gives the training loss as a mean-squared regression:

$$\mathcal{L}_{\mathrm{FM}} \;=\; \mathbb{E}_{t,\,x_0,\,x_1} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2 .$$

In practice, every training step samples a time $t \in [0, 1]$ and a noise-data pair (x_0, x_1), computes the interpolant x_t, and takes a gradient step on this squared error.

Flow matching with this linear interpolant generalises score-based diffusion: the variance-preserving Gaussian-path objective of denoising diffusion models [1] is a special case of the FM family [2, 4].

Generating samples from a trained flow-matching model amounts to integrating the learned velocity field. This is often performed with a standard ODE solver, with Euler steps usually sufficing for stable trajectories. The field also admits stochastic sampling variants — such as those drawing on the connection to score-based stochastic differential equations (SDEs) — which inject noise during the integration steps to explore the probability space and improve sample diversity. The models we work with in this report, Tabasco and Proteina, support both deterministic and stochastic samplers.

Conditional generation augments v_θ with context tokens — such as a class label, a sequence length, or a structural motif — with conditioning supplied at both training and sampling time. In this report, we focus on unconditional generation only.

For a more complete treatment of flow matching, we refer the reader to Lipman et al. [2] for the original presentation, and to the recent Lipman et al. guide [4].

2.2 Representation alignment

Chapter 1 introduced the argument made by Yu et al. [5] about the *representation bottleneck* in diffusion-transformer models. Their analysis applies to flow-matching models as well, given the equivalence between the two objectives discussed above (§2.1). The flow-matching objective implicitly asks a model to learn two things at once: a useful representation of the data, and a velocity field that interprets this representation. This twin burden, they argue, leads to slow convergence and large training budgets.

Their proposed solution, Representation Alignment (REPA), addresses the first half of this burden by adding a regularisation term to the training loss. Figure 2.1 shows the construction. The regularisation pulls z_ℓ, the generator's intermediate hidden state at a chosen *alignment layer* ℓ, towards the embeddings produced by a frozen pretrained encoder f_{enc} run on the clean data sample x_1. The hidden state z_ℓ is a sequence of N token vectors. In the context of images, each token represents an image patch; in the molecular

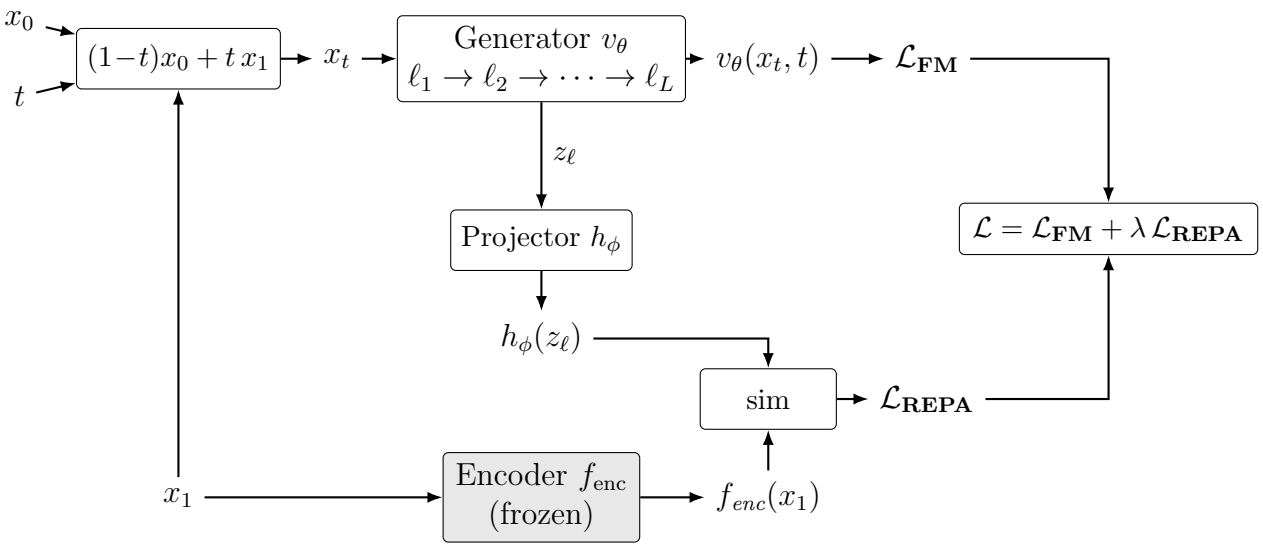

Figure 2.1: The REPA construction. A noised sample $x_t = (1-t)x_0 + t\,x_1$ is constructed at each training step from a noise sample x_0, a clean data sample x_1, and a sampled timestep $t \in [0, 1]$. The generator v_θ processes x_t through its sequence of layers $\ell_1, \ldots, \ell_L$ to produce the velocity field that feeds the flow-matching loss $\mathcal{L}_{\text{FM}}$. Separately, the clean sample x_1 is also passed through a frozen pretrained encoder f_{enc} (shaded). The generator's hidden state z_ℓ at the chosen alignment layer is mapped into the encoder's feature space by a trainable projector h_ϕ; the REPA loss $\mathcal{L}_{\text{REPA}}$ is the negated similarity between $h_\phi(z_\ell)$ and $f_{\text{enc}}(x_1)$, averaged across tokens. The total training loss $\mathcal{L}$ combines them with weight λ.

setting, each token represents an atom or residue that is part of the larger molecule (§2.3). A small trainable *projector head* h_ϕ — a three-layer multi-layer perceptron (MLP) in the original paper — maps z_ℓ into the encoder's feature space, so that $h_\phi(z_\ell)$ and $f_{\text{enc}}(x_1)$ can be compared. This alignment is asymmetric. The encoder provides a fixed target throughout training, while the generator parameters θ and the projector ϕ are updated.

The alignment loss is the negated similarity, averaged token-wise, between projected and target embeddings:

$$\mathcal{L}_{\text{REPA}} \;=\; -\,\mathbb{E}_{t,\,x_0,\,x_1}\left[\frac{1}{N}\sum_{n=1}^{N}\text{sim}\big(h_\phi(z_\ell)_n,\; f_{\text{enc}}(x_1)_n\big)\right],$$

where n indexes the N tokens within a sample. In the image setting, N is constant across the dataset, so this token-wise average is equivalent to averaging similarities once per sample and then across the batch. Indeed, the original paper draws no distinction between the two. For molecular data, where N varies per sample, the two diverge. We return to this distinction in the design knobs below.

The final training objective is the flow-matching loss from §2.1 plus this term, scaled by a weight λ:

$$\mathcal{L} \;=\; \mathcal{L}_{\text{FM}} \;+\; \lambda\,\mathcal{L}_{\text{REPA}}.$$

This construction exposes several design choices, some of whose effects we revisit in the

ablations of Chapter 6:

- The choice of encoder f_{enc}. Different encoders may capture distinct aspects of the data distribution.
- The alignment layer ℓ at which we attach the projector. Yu et al. experimentally determined that mid-network layers work best for image diffusion, but the optimal choice is not obvious a priori.
- The alignment-loss weight λ, which balances the alignment signal against the generative objective. Once again, Yu et al. experimentally determined an optimal value of 0.5 for image diffusion, but this may not hold universally.
- The choice of similarity metric sim. Yu et al. compared cosine similarity and mean-squared error for image diffusion, and found the former more effective. We adopt cosine similarity in this report as well.
- The averaging mode. Averaging tokens within each sample first weights every molecule equally, while pooling tokens across the batch gives larger molecules proportionally more REPA signal.

We adopt the core REPA construction as in Yu et al. for this report. However, a small family of methodological variants on REPA has since emerged. Dispersive regularisers [12] replace the external encoder with an internal contrastive term, while flexible-target formulations [7] broaden which encoder features the loss attaches to. Separately, REPA-style alignment has been applied beyond the image setting, to video [9], audio [10], materials [11], and molecules [8]. We defer exploration of these variants to future work.

2.2.1 What makes for a good REPA encoder?

There are many possible choices for the encoder f_{enc} in any REPA construction, and this selection is a key design decision. But what characteristics make for a good alignment target? In their original report, Yu et al. [5] observed that ImageNet linear-probe accuracy correlates positively with generation performance, suggesting that global semantic representation is the driving factor. However, more recent work by Singh et al. [6] reports the opposite after profiling a wider range of encoders. Linear-probe accuracy does not in fact predict REPA gain, they argue, while a measure of the spatial structure of patch-token representations does. This suggests that generation performance for REPA may be driven by *local* feature geometry, rather than *global* semantic strength. Encoder characterisation for REPA remains an open research question, particularly in the molecular generation setting, and we pick up the thread in Chapter 4.

2.3 A primer on molecular generation

Having outlined the modelling paradigms driving our report, we now turn to the molecular setting in which we apply them. To situate our work, we begin with a brief primer on

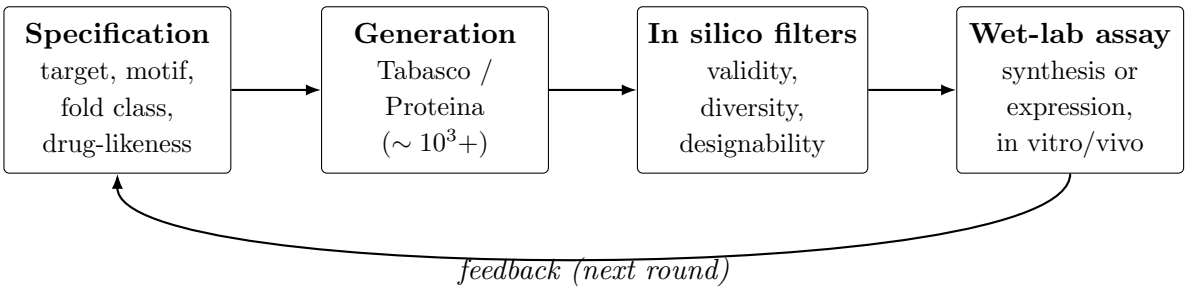

Figure 2.2: The de novo molecular design cycle. An upstream specification drives a generative model to produce a candidate pool, which is funnelled through in silico filters and then synthesised (or expressed) and assayed in the wet lab. Experimental results feed back into the next round of generation. Typical generation-step pool sizes are in the thousands to tens of thousands of candidates per round [18].

the role generative models play within the broader de novo molecular design pipeline. Generative models are rarely intended to produce ready-to-use artefacts. Instead, they act as high-throughput in silico proposal engines in an iterative process of refinement and selection. This process, often called a *design campaign* [50], is summarised in Figure 2.2.

An upstream specification, such as a drug-likeness profile or a fold class, conditions a generative model to produce a pool of candidate samples. These pools typically comprise thousands to tens of thousands of candidates per round [18], which are then funnelled through in silico filters. For small molecules, these typically include checks for structural validity and physical plausibility [51], drug-likeness [52], and target docking; for protein backbones, designability and structural diversity are the dominant considerations [14]. The select few survivors are produced in a laboratory, through chemical synthesis (small molecules) or expression in cell lines (proteins), and evaluated using in vitro or in vivo assays. Experimental results then feed back into the next round of generation.

This pipeline framing is relevant to our discussion of evaluation metrics in Chapter 3. Several metrics we adopt, such as structural validity and designability. are not measures of intrinsic molecular quality, but proxies for specific in silico filter steps.

2.4 Tabasco and Proteina

The two models we study — Tabasco [15] for small molecules and Proteina [14] for protein backbones — are both non-equivariant transformer flow-matching generators. They are explicitly designed to scale with data and model size, rather than relying on structural priors imposed by model architecture. The rotational and translational symmetries expected of 3D molecules are intended to be learned from data and augmentation rather than imposed by construction.

Tabasco is motivated by the success of similar architectures in protein-structure generation, like Proteina, setting it apart from equivariant message-passing small-molecule generators such as EQGAT-diff [45], MiDi [46], and SemlaFlow [47]. In turn, Proteina

is motivated by the pair-bias attention mechanism introduced in AlphaFold 3 [41], which distinguishes it from SE(3)-equivariant backbone generators such as RFdiffusion [18], FrameFlow [43], and Genie [44]. Despite their architectural minimalism, both achieve performance comparable or superior to equivariant baselines on structural-quality evaluations.

2.4.1 Small molecules and Tabasco

In medicinal chemistry, a *small molecule* is a drug-like compound at the sub-900-Dalton scale [48]. In silico, its structure is typically represented using two coupled features: a discrete *molecular graph* of typed atoms connected by typed bonds, and a continuous *conformer* placing those atoms in 3D Cartesian space. A flow-matching parameterisation must therefore handle both jointly.

Tabasco [15] represents each molecule as a sequence of per-atom tokens, each carrying an atom-type embedding and a 3D coordinate, but with no explicit bond decoder. This treats the molecule as a point cloud of typed atoms rather than a graph, sidestepping edge modelling at the cost of having to recover bond structure post hoc from interatomic distances. Tabasco is trained on GEOM-drugs [32], a public dataset of drug-like conformers.

This point-cloud-of-atoms parameterisation is one of several options in the literature, and each makes a different trade-off. *SMILES* and *SELFIES* are string encodings that work well with autoregressive models but discard the 3D conformer entirely; we encounter them later as the parameterisation employed by some candidate encoders we evaluate in Chapter 4. *3D voxel grids* preserve geometry, but lose the discrete graph and inflate the token count by an order of magnitude or more. *Equivariant point clouds*, used by generators such as EQGAT-diff [45], MiDi [46], and SemlaFlow [47], keep both graph and geometry, but pay the computational cost associated with SE(3)-equivariant layers. We refer the reader to Duval et al. [36] and Wang et al. [22] for broader surveys in this space.

2.4.2 Protein backbones and Proteina

A *protein* is a chain of amino-acid residues, ranging from around 50 to several thousands in length [49]; the largest known, titin, has approximately 34,000 residues. (Shorter chains are typically classified as *peptides* instead.) Each amino-acid residue contributes four *backbone* atoms (N, C_α, C, O) whose interactions determine the chain's *fold* geometry, and a *side chain* that determines its amino-acid identity.

For the structure-generation problem we study, it is customary to only model the backbone, with sequence identity recovered downstream using a separate *inverse-folding* model such as ProteinMPNN [24]. A further compression is the *CA-trace*, which uses only the set of C_α coordinates for every residue to represent the entire protein. This parameterisation retains enough information to recover fold topology and reduces computational cost relative to the more complete representation, which is typically referred to as *all-atom*

modelling. Proteina [14] adopts this C_α parameterisation, representing each backbone as a sequence of per-residue tokens carrying continuous C_α Cartesian coordinates. The models reported in the original presentation are largely trained on high-confidence synthetic structures from the AlphaFold Database (AFDB) [31].

Once again, the CA-trace parameterisation is one of several techniques used in the literature to parameterise proteins in silico. *Sequence-only* models, such as the ESM lineage [23], operate on amino-acid identity strings. These models capture rich functional priors, but encode far less information on 3D structure. *Full-atom* representations close the gap to physical realism, but add substantial compute cost. *Equivariant* parameterisations, such as torsion angles and oriented residue frames, encode SE(3) symmetries by construction, and form the standard basis for the equivariant generators noted above. However, these architectures have historically bottlenecked scaling. We refer the reader to Notin et al. [53] for a broader survey of machine-learning approaches to protein design.

2.5 Open questions

This survey leaves two questions open that the rest of this report takes on:

(i) *Does REPA work in the molecular domain?* Existing applications of REPA in molecular generation [7, 8] have not been ablated over encoder choice, and protein backbone generation in particular remains unexplored.

(ii) *When does REPA work?* The encoder-characterisation work surveyed in §2.2.1 has begun to ask which properties of an encoder predict REPA gain, but only within the image setting. We extend this question to the molecular domain.

We take both questions up empirically in Chapters 4–6.

Chapter 3

Evaluating molecular generation

Chapter 4

Encoder targets and profiling

Chapter 5

REPA on small molecules

Chapter 6

REPA on protein backbones

In this chapter, we expand the scope of our study on representation alignment from small molecules to much larger ones: proteins. Previously (Chapter 5), our efforts using REPA to generate small molecules yielded little demonstrable improvement, largely because the Tabasco [15] baseline was already saturated on most measures of generation quality. However, de novo backbone design represents a more challenging task, as the data distribution of proteins is significantly larger, and the geometry of these structures is correspondingly more complex. In this context, we ask: *can REPA accelerate or improve the training of Proteina [14], a flow-matching transformer-based protein backbone generator?*

Our headline finding is that REPA accelerates convergence on the two key measures of protein generation quality, in both synthetic and experimental training-data regimes (Figure 6.1). This chapter characterises *what* happens and attempts to understand *why*. We also pick up the thread from Chapter 3 on how the molecular domain differs from the image domain REPA was built for, and develop a theme throughout: REPA here is less a single mechanism than a *family* of interventions.

6.1 Experimental setup

We integrate REPA into Proteina [14], a 60M-parameter non-equivariant flow-matching generator of Cα protein backbones. Following §2.2, we attach a trainable projector at a chosen trunk layer that aligns the model's hidden states to a frozen structural encoder.

6.1.1 Datasets

We train in two regimes that span the field's experimental–synthetic spectrum: the experimentally-determined Protein Data Bank [30] and the AlphaFold-predicted AlphaFold Database [31] (Swiss-Prot subset; Table 6.1). We use the manually-curated Swiss-Prot subset of AFDB as a reproducible stand-in for the Foldseek-clustered TrEMBL subset on which Proteina was trained [14] (whose released structure index references UniProt accessions since pruned from AFDB); both are AlphaFold-predicted, so the experimental-versus-synthetic contrast is unaffected.

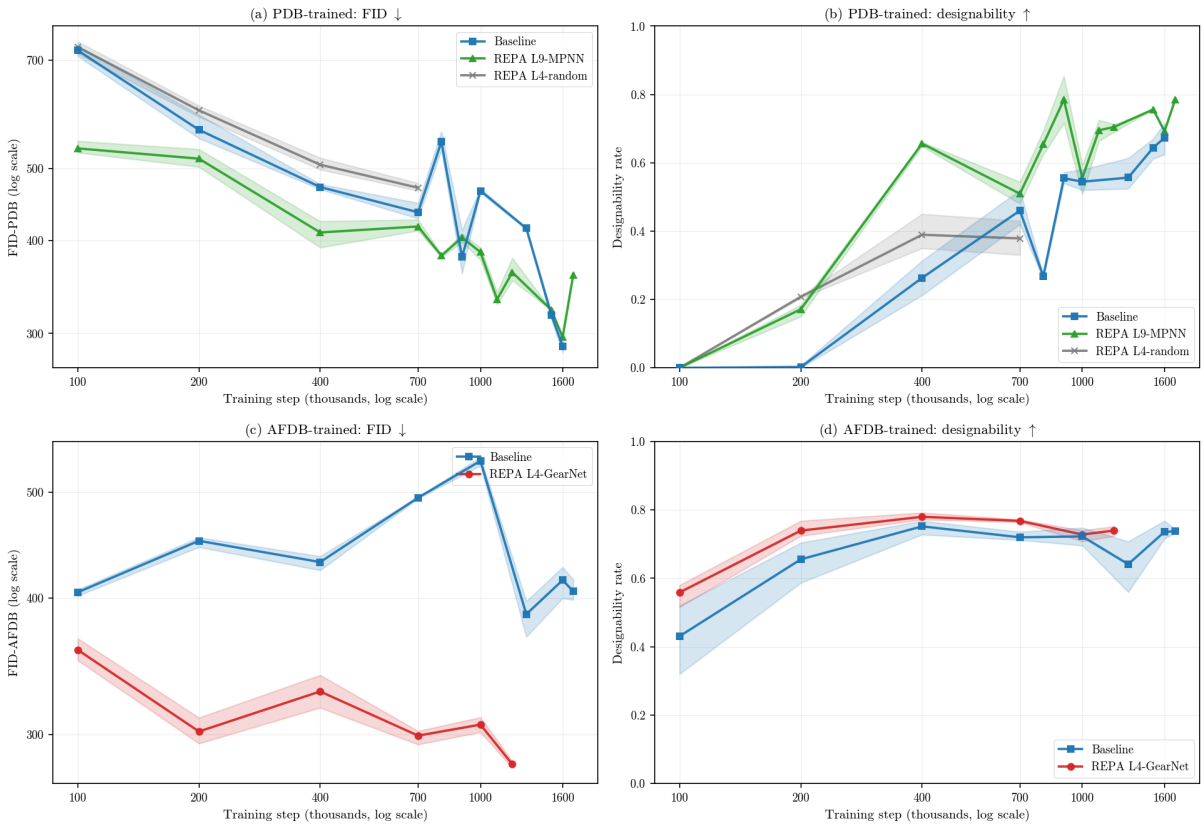

Figure 6.1: **REPA accelerates both FID and designability, in both PDB (experimental) and AFDB (synthetic) data regimes.** The random-encoder control (grey) stays close to the baseline while the learned encoders open a clear gap. This indicates that the acceleration is a mechanistic effect of the encoder's learned structure, rather than of the regularisation induced by alignment loss alone. While PDB retains a persistent designability gap, its baseline appears to catch up on FID after ≈1M steps; we examine this in §6.4. (n=256 models, 3 seeds; shaded bands show min/max.)

Different datasets in this domain carry different biases that must be accounted for. Trends in designability offer a clear example, as training on AFDB raises the in-silico-designability floor well above PDB at every setting we tested (§6.4). The likely cause is circular. The canonical designability test inverse-folds a generated backbone with ProteinMPNN [24] and re-folds it with ESMFold [23] (Chapter 3). Both of these models share their lineage with the AlphaFold2 [40] network that produced AFDB in the first place, so AlphaFold-like backbones are plausibly the ones it re-folds most confidently. AFDB backbones are also more idealised (low-clash, high-confidence, mostly single-domain), and thus intrinsically more designable. We do not test these mechanisms, but flag this observation so metrics are read with care. We return to this theme in Chapter 7.

	PDB	AFDB (Swiss-Prot)
Origin	Experimental (X-ray, cryo-EM)	Predicted (AlphaFold2, v6)
Per-residue confidence	B-factor	pLDDT
Train / val / test chains	273,771 / 3,190 / 323	229,670 / 4,521 / 235
Length filter	$\leq$256 res	$\leq$256 res
CATH coverage (train)	44.2% (α/β-dom.)	61.8% (α/β-dom.)
Val$\leftrightarrow$train, $\geq$30% id.	98.3%	92.2%
AFDB-val$\leftrightarrow$PDB-train, ident.	5.7% (vs 79% in-DB)	

Table 6.1: **Our two regimes differ by origin — experimentally-determined (PDB) versus AlphaFold-predicted (AFDB) — which is the contrast we exploit, not raw scale.** Counts are for the headline n=256 regime in a random 98/1.9/0.1 split (the n=128 scale check refilters the same pools to $\leq$128). Neither split is sequence-clustered, so throughout we read rank-order and Δ-from-baseline rather than absolute in-house scores (§3); the cross-DB row (AFDB validation chains identical to a PDB training chain) motivates the leakage-clean AFDB probe set used there.

6.1.2 Models

Our experiments vary the two design axes that Chapter 4 flagged as the most consequential for REPA and the least explored in the molecular setting: the encoder target and the injection depth (Table 6.2). We study the two encoders profiling identified as viable — CA-GearNet [25], which encodes global contact-graph topology, and ProteinMPNN [24], which encodes each residue's local inverse-folding environment — each attached at two trunk depths, layer 4 and layer 9 of ten. A random-weights GearNet, identical in architecture but never trained, is our falsifier control: it separates whatever REPA achieves merely by adding an auxiliary loss from what it achieves through the encoder's *learned* structural knowledge. **That two such different encoders are both viable targets is why REPA here behaves as a *family* of interventions: each alignment target addresses a different gap in the core architecture and routes the generation gain to a different aspect of quality.**

6.1.3 Hyperparameters

Unless stated otherwise, we hold the alignment and evaluation settings fixed at the defaults in Table 6.3: alignment weight $\lambda = 0.5$ with cosine similarity averaged per residue, residue length $n = 256$ as the headline regime ($n = 128$ as a scale-robustness check, §6.8), both data regimes reported, and the Chapter 3 supercolumn suite evaluated at the default SDE sampler noise level $\gamma = 0.45$.

Component	Setting
Generator	Proteina (CAFlow), 60M parameters
Trunk	10-layer non-equivariant transformer; token dim 512, 8 heads
Parameterisation	raw Cα coordinates (no architectural symmetry priors)
Alignment loss	cosine similarity, averaged per residue (Table 6.3)
Projector	MLP, hidden dim 512 (depth in Appendix A)
Injection depth ℓ	layer 4, layer 9 (of 10; layer 0 in the fuller sweep)
Encoder targets	CA-GearNet (512-d, frozen) · ProteinMPNN · random-GearNet (control)

Table 6.2: **Model and REPA configuration: a 60M non-equivariant trunk, with encoder target and injection depth as the two axes under test.** The two axes we vary — encoder target and injection depth — give four learned configurations ({GearNet, MPNN} $\times$ {L4, L9}) plus the random-GearNet falsifier, trained in both regimes; the broader layer/encoder sweep and all optimisation hyperparameters (batch size, learning rate, step counts) are in Appendix A, and the held-fixed training/evaluation defaults in Table 6.3. The non-equivariant trunk makes Proteina a clean testbed: any structural inductive bias it acquires is learned, not imposed.

Hyperparameter	Default (unless stated)	Varied / swept in
Alignment weight λ	0.5 (after Yu et al. [5])	Appendix A
Similarity	cosine	—
Token averaging	per residue	§2.2 (design knob)
Residue length n	256 (headline regime)	128 (§6.8)
Training regime	PDB *and* AFDB (both reported)	—
Sampler	SDE, noise scale $\gamma = 0.45$	§6.6
Evaluation	supercolumn suite (Table 6.4)	—

Table 6.3: **Held-fixed training and evaluation hyperparameters: our "unless stated otherwise" defaults.** The alignment weight follows the value Yu et al. [5] found effective for image diffusion; cosine similarity averaged per residue is the REPA objective of §2.2, with the per-residue (vs. per-sample) averaging mode being a design knob we discuss there. We anchor headline claims at residue length n=256 and use n=128 as a scale-robustness check (§6.8). Every variant is trained in both data regimes and sampled at the default SDE noise scale γ=0.45, which we ablate across six values in §6.6. Optimisation hyperparameters (batch size, learning rate, step counts) and the λ/batch-size/projector-depth ablations are in Appendix A.

6.1.4 Metrics

We measure three things (Table 6.4). *Representation quality* is the linear-probe recoverability of structural labels from the trunk (the probe principle of §3): a per-residue battery (inverse folding and backbone dihedrals) and a per-chain CATH fold probe, for which we report *architecture* (CATH-A) as the headline level — Class is too coarse to discriminate and Topology too finely-bucketed to probe reliably (Table 6.5). *Representation alignment* is CKNNA [56], the mutual-nearest-neighbour kernel-alignment between the trunk and an encoder. *Generation quality* is the Chapter 3 supercolumn suite — quality, tertiary-whole (T-W), tertiary-designable (T-D), secondary-whole (S-W) and secondary-

Metric	What it captures	Granularity
Representation quality — linear-probe recoverability of structural labels from the trunk		
IF top-1↑	inverse-folding sequence recovery (semantic content)	per residue
Dihedral MAE↓	backbone dihedral-angle error (low-level geometry)	per residue
CATH-A↑	fold-architecture accuracy — headline fold metric (Table 6.5)	per chain
Representation alignment		
CKNNA↑ [56]	mutual-nearest-neighbour kernel alignment between trunk and encoder	per residue
Generation quality — the supercolumn suite (Chapter 3)		
Quality	per-sample designability (Des↑) and FID↓ to the reference set	per set
T-W (tertiary, whole)	whole-set fold-distribution match and coverage (fJSD-A↓, fS-A↑, pwTM↓)	per set
T-D (tertiary, designable)	fold diversity within the designable subset (#Clust↑, pwTM↓)	per set
S-W (secondary, whole)	whole-set secondary-structure distribution match (ssJSD-2D↓)	per set
S-D (secondary, designable)	secondary-structure match and β-content of the designable subset (ssJSD-2D↓, E%des↑)	per set

Table 6.4: **We measure three things: representation quality, representation alignment, and generation quality.** Representation quality is the linear-probe recoverability of structural labels from the trunk — a per-residue battery (inverse-folding sequence recovery and backbone dihedral error) and a per-chain fold probe on CATH, for which we report *architecture* (CATH-A) as the headline level (Table 6.5). Representation alignment is CKNNA [56], the mutual-nearest-neighbour kernel-alignment between a trunk layer and an encoder. Generation quality is the Chapter 3 supercolumn suite, which decomposes into per-sample *quality*, whole-set vs. designable-subset *tertiary* structure (T-W, T-D), and whole-set vs. designable-subset *secondary* structure (S-W, S-D); arrows give the good direction for each member metric, and the full per-metric breakdown appears in Tables 6.8–6.10.

designable (S-D) — which we use to anatomise the effect in §6.5.

6.2 REPA installs structural representations the baseline never learns

Flow matching alone never teaches Proteina to represent structure. We probe the trunk's hidden states for structural labels — inverse folding, backbone dihedrals, and CATH fold class [55] — across training. The baseline is flat on every probe for 1.8M steps (Figure 6.2). It learns to *generate* backbones without ever learning to *represent* them.

REPA installs that structure, and as a permanent gap. Every learned variant lifts the probes far above the baseline and never gives the lead back. Fold-topology accuracy

CATH level	#classes	baseline probe acc.	verdict
C (class)	4	0.733	one class (α/β) is 55% of the data; coarse, already near-saturated — *gameable*
A (architecture)	∼40	0.407	balanced and discriminative, with headroom to move — **our headline fold metric**
T (topology)	∼1400	0.301	long-tailed (top fold ∼18%; only ∼89 classes populated enough to probe); per-class accuracy is unreliable — *too many buckets*

Table 6.5: **We read fold-level representation quality off CATH *architecture* — Class is too coarse to discriminate, Topology too finely-bucketed to probe reliably.** Class has 4 buckets (one at 55%, baseline already 0.73); Topology has ∼1400 imbalanced buckets; Architecture sits between — enough classes to be meaningful, enough examples per class to be learnable — so CATH-A is our headline fold metric (C and T are reported in the appendix). ("Baseline probe acc." is a linear probe on the un-aligned trunk.)

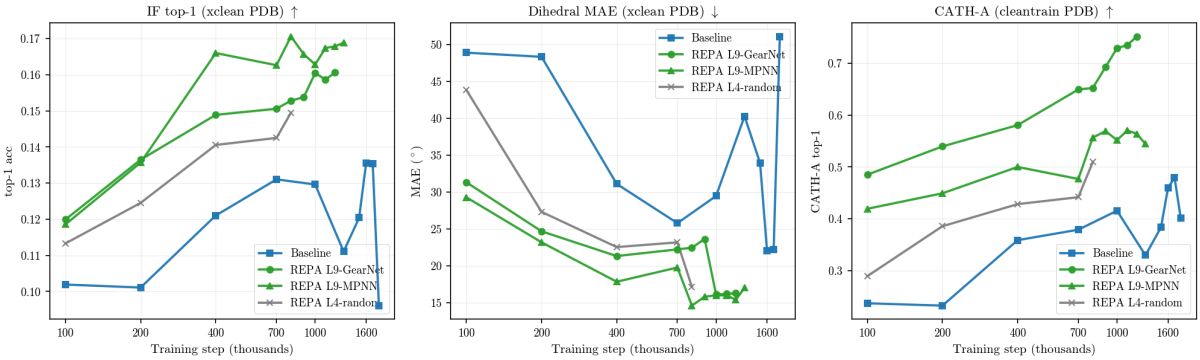

Figure 6.2: **Flow-matching alone never makes the student's representations more structural; REPA installs it as a persistent, encoder-axis-specific gap.** The baseline stays flat for 1.8M steps while every REPA variant opens a gap that never closes. The rank-order is axis-specific — GearNet leads the fold probe (CATH), MPNN the per-residue probes (inverse folding, dihedral) — and the random control gains least, so the effect draws on the encoder's learned structure, not the auxiliary loss alone. (n=256, PDB-trained; best-layer linear probes on the doubly-clean cross-database slice, §3.)

rises from 0.30 to 0.75 under L9-GearNet. This is a gap, not a head start — unlike the generation side (§6.4), where the baseline eventually catches up.

The gain is encoder-routed, and the routing is axis-specific. GearNet wins the fold probes; MPNN wins the per-residue probes (Table 6.6). Global structure goes to GearNet, local geometry to MPNN. This is exactly what a multi-factorial representation predicts (§3): no single encoder captures every factor, so the encoder you pick chooses the factor REPA improves.

The effect needs a trained encoder, not just an auxiliary loss. The random-

	IF top-1↑	dihedral MAE (°)↓	CATH-A↑	CATH-T↑
Baseline	0.123	32.1	0.407	0.301
REPA L4-random	+0.023	−11.9	+0.069	+0.077
REPA L9-GN	+0.033	−12.6	+0.295	+0.451
REPA L9-MPNN	+0.044	−15.7	+0.141	+0.172

Table 6.6: **REPA's representation gain is encoder-routed: GearNet wins the fold probes (CATH), MPNN the per-residue probes (IF, dihedral), and the random control gains least — so the effect needs the encoder's learned structure, not just an auxiliary loss.** (n=256 PDB, best-layer probes, mean over steps ≥700K; baseline absolute, REPA rows Δ from baseline; green = improvement, darker = best per probe.)

weights control improves least on every probe. Its gain is small-but-positive rather than zero — an untrained graph encoder still imposes weak geometric structure — a thread we pick up at the mechanism (§6.5) and at smaller scale (§6.8).

6.3 REPA pulls the trunk's geometry towards the encoder's

Beyond decodability, REPA moves the trunk's geometry onto the encoder's. These are different claims; we measure the second with CKNNA [56], per residue, between each trunk layer and the encoder. The baseline sits at the noise floor at every layer. REPA-GearNet lifts alignment 20–25-fold, peaking at layer 8 and collapsing at the velocity-output layer (Figure 6.3). Because the baseline never moves, the entire signal is REPA's — a cleaner causal story than the image-domain original.

Aligning to one encoder aligns the trunk to all of them. A GearNet-aligned trunk moves closer to ProteinMPNN and ESM2 [23] as well — the off-diagonal signature of Platonic convergence [56], and a rebuttal to the worry that aligning CKNNA to one's own target is circular. ESM2 is the strongest attractor of all. The gains are strictly per-residue: the per-protein matrix is flat, which keeps us from over-claiming a fold-level benefit.

6.4 REPA accelerates generation, durably on synthetic data

On synthetic data the acceleration is large and permanent. REPA-GearNet reaches the AFDB baseline's best-ever FID 6.5–13× earlier and keeps improving past it (Table 6.7); the baseline floors at 386 and never closes the gap. *One caveat:* AFDB's designability edge is partly a proxy artefact — the ProteinMPNN→ESMFold round-trip shares lineage with the AlphaFold2 that built AFDB (§6.7) — but the FID, distribution-match, and representation gains share no such lineage.

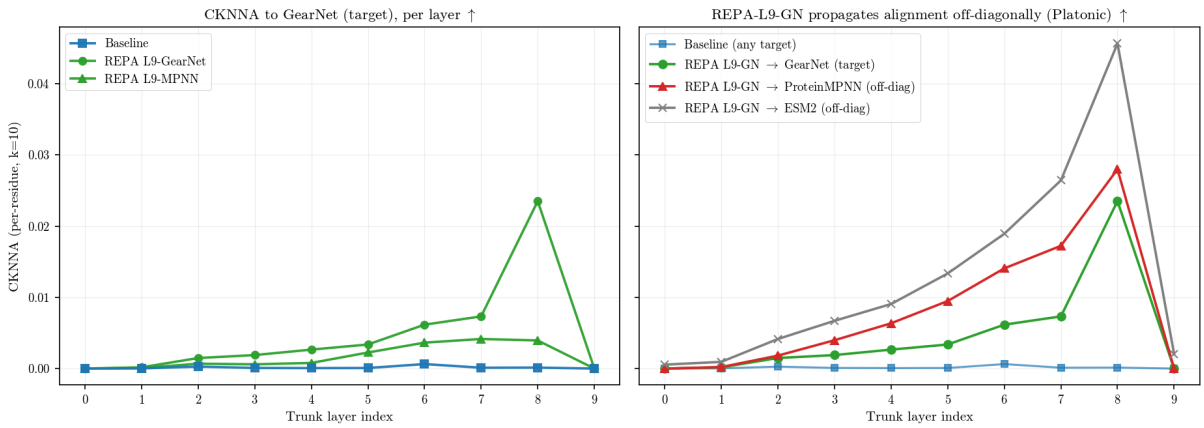

Figure 6.3: **REPA pulls the trunk's geometry toward the encoder's — and toward encoders it was never trained on (Platonic convergence).** The baseline sits at the noise floor at every layer; REPA lifts per-residue alignment ≈ 20–25×, peaking mid-stack and collapsing at the output layer. Aligning to GearNet alone also pulls the trunk toward ProteinMPNN and ESM2 — the off-diagonal Platonic signature, with ESM2 the strongest attractor. The gains are per-residue: the per-protein matrix is flat, so we do not over-claim a fold-level benefit. (n=256 PDB, step 1M, t=1.0; per-residue CKNNA, k=10.)

On experimental data the acceleration is real but transient. REPA-L9 leads on FID at every step up to 1.2M, by 27% at 700K. But the PDB baseline is a strong asymptotic learner: it matches REPA's plateau by ≈1.4M and edges below by 1.6M. On designability the PDB story is cleaner — a 1.8–2.3× speedup, with the random control never reaching the baseline's best.

The regime split is about the baseline, not REPA. The AFDB baseline plateaus poorly, so REPA's head start is never given back; the PDB baseline is strong, so it closes. Same intervention, opposite asymptotics — REPA helps most where the base learner struggles most. It is also the first sign that REPA's gains are front-loaded, a theme we return to at the trade-off [13].

The generation gain plausibly runs through representation. Across checkpoints, better student geometry tracks better generation, and the two move together only under REPA (Figure 6.4). Lower dihedral error predicts higher designability (Pearson $r = -0.54$; -0.47 controlling for training step). The same envelope holds for inverse folding, and on AFDB for fold accuracy against FID.

The link is encoder-matched, which is the strongest evidence that it is causal. GearNet's fold-representation advantage surfaces as a fold-*distribution* gain; MPNN's local advantage surfaces as a *designability* gain. Both sides rank-order the encoders identically. We stop short of formal mediation — the random control brackets the relationship, but co-movement is not proof.

Dataset	Metric	Variant	Baseline best	REPA reaches @	Speedup	REPA's own best
FID acceleration — durable on AFDB:						
AFDB	FID-AFDB	L4-GN	386 @1.3M	100K	13.0×	282
AFDB	FID-AFDB	L9-GN	386 @1.3M	200K	6.5×	313
AFDB	FID-AFDB	MPNN-L9	386 @1.3M	400K	3.2×	352
AFDB	FID-AFDB	MPNN-L4	386 @1.3M	never	—	416
FID acceleration — transient on PDB (baseline catches up late):						
PDB	FID-PDB	L9-GN	∼330 plateau	700K	∼1.3–1.5×	319 (@1.2M)
PDB	FID-PDB	L9-MPNN	∼330 plateau	1.1M	∼1.3×	334 (@1.1M)
PDB	FID-PDB	MPNN-L4	329 plateau (1.6M)	1.6M	budget-matched	329 (@1.6M)
Designability acceleration — clean on PDB:						
PDB	Des	L4-GN	0.67 @1.6M	700K	2.3×	0.70
PDB	Des	L9-MPNN	0.67 @1.6M	900K	1.8×	0.81
PDB	Des	L4-random	0.67 @1.6M	never	—	0.39
AFDB	Des	(all)	baseline saturates at 0.75 by 400K — acceleration not meaningful (proxy bias, §6.4)			

Table 6.7: **REPA's acceleration is durable on AFDB FID (6.5–13×) and clean on PDB designability (1.8–2.3×), but only transient on PDB FID.** On AFDB, REPA-GearNet keeps improving past the baseline's FID floor; on PDB the FID lead is handed back as the long baseline catches up, yet REPA's designability advantage persists — the random control never reaches the baseline's best. ("Baseline best" = baseline's cumulative-best value; "REPA reaches @" = first step REPA matches it; speedup = their ratio. n=256.)

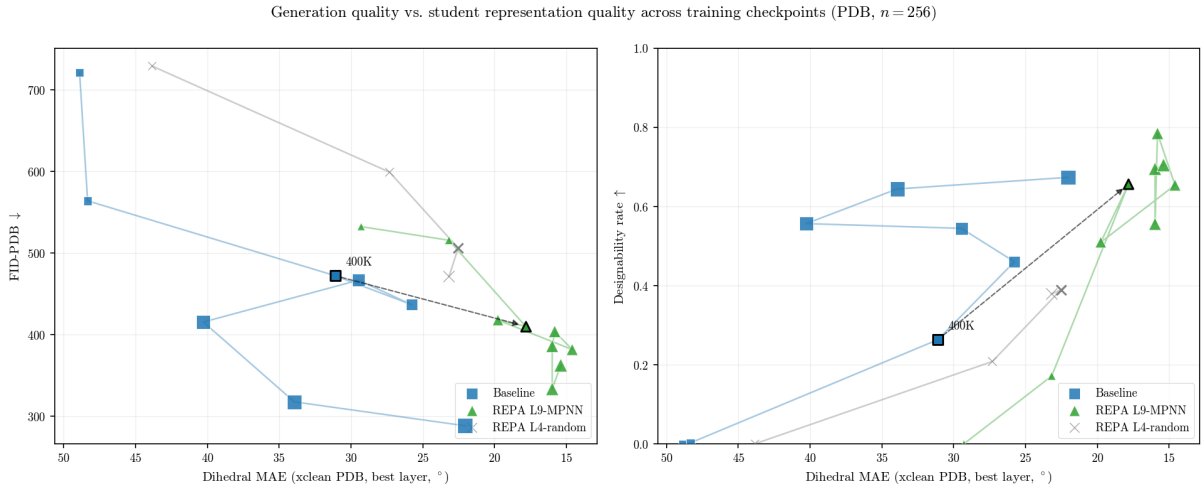

Figure 6.4: **Better student geometry buys better generation at equal compute — direct evidence REPA relieves the representation bottleneck.** Baseline, REPA L9-MPNN, and the random control all fall on a common band: lower student dihedral error accompanies better generation on both FID and designability. At matched compute (≈400K steps, dashed arrow) REPA sits further along that band than the baseline. (n=256 PDB, one marker per checkpoint; x-axis student dihedral MAE, better to the right; left FID-PDB ↓, right designability ↑.)

Variant	Quality		T-W			T-D		S-W	S-D	
	Des↑	FID↓	fJSD-A↓	fS-A↑	pwTM↓	#Clust↑	pwTM↓	ssJSD2D↓	ssJSD2D↓	E%des↑
PDB-trained, n=256, 700K, 3 seeds; FID is FID-PDB (in-distribution).										
Baseline	0.46	437	1.11	5.48	0.22	77	0.27	0.35	0.40	0.22
REPA L4-random	−0.08	+34	−0.55	+0.05	−0.05	−23	−0.10	−0.13	−0.01	−0.11
REPA L4-GN	+0.24	+71	−0.38	+1.63	+0.13	+1	+0.11	−0.21	−0.12	+0.00
REPA L9-GN	+0.14	−118	−0.57	+2.31	−0.03	+27	−0.02	−0.19	−0.10	−0.04
REPA L4-MPNN	+0.18	−88	−0.13	+0.48	+0.05	−4	+0.08	−0.16	−0.14	+0.01
REPA L9-MPNN	+0.05	−19	−0.32	+0.37	−0.01	+33	−0.01	−0.14	−0.11	−0.07
AFDB-trained, n=256, 700K, 3 seeds (MPNN-L4: 1 seed); FID is FID-AFDB.										
Baseline	0.72	494	2.45	4.59	0.20	128	0.22	0.15	0.16	0.15
REPA L4-GN	+0.05	−195	−1.56	+2.27	+0.00	−35	−0.04	−0.11	−0.07	−0.02
REPA L9-GN	+0.01	−172	−1.29	+2.76	+0.01	−50	+0.11	−0.12	−0.06	+0.04
REPA L4-MPNN	−0.03	−22	−0.39	+0.03	—	−1	−0.07	−0.02	+0.05	−0.01
REPA L9-MPNN	+0.05	−139	−0.93	+0.31	−0.03	+21	−0.03	+0.01	+0.02	−0.04

Table 6.8: **At one fixed early step, REPA's effect is already encoder-routed — GearNet to whole-set distribution, MPNN to per-sample quality.** GearNet wins the tertiary-whole metrics (FID, fJSD-A, fS-A) on both datasets; MPNN wins quality (Des). The random control wins whole-set distribution-match but hurts quality and designable diversity — a regulariser without the quality benefit. (n=256, step 700K; baseline absolute, REPA rows Δ; green = improvement, red = regression; header arrows give the good direction.)

6.5 REPA improves whole-model coverage but trades off designable diversity

REPA's effect on generation is not uniform; the supercolumn suite resolves it. Decomposed across quality, tertiary, and secondary structure (Table 6.8), the gain is strongest and most encoder-robust on whole-set distribution, encoder-specific on per-sample quality, and — the load-bearing finding — carries a cost in designable diversity that grows with training.

Whole-set distribution and diversity improve under almost every variant. Fold entropy rises, and secondary-structure distribution-match is the single most robust REPA effect — it holds even for the random control. This is the generic core: aligning to any reasonable structural encoder makes the generated *set* match the reference better.

Per-sample quality and secondary-structure direction are encoder-specific. MPNN accelerates designability on both datasets; GearNet helps it less consistently — the generation-side mirror of the representation rank-order. The designable subset also shifts in secondary-structure composition, but the *direction* is encoder×dataset gated: β-ward for learned REPA on PDB, α-ward for MPNN on AFDB. It is not a generic "more sheets" effect.

The cost is concentration: REPA funnels designable samples onto a few encoder-preferred, β-rich folds. Whole-set fold coverage rises while distinct designable folds fall. The loss is localised to the sheet-rich folds, where it is severe (Table 6.9): designable pwTM in the $\beta \geq 25$ bin climbs from the baseline's ≈0.13 to 0.87 under GearNet.

Designable-subset pwTM↓	700K steps			1.0M steps		
	β<10	10–25	$\beta\geq$25	β<10	10–25	$\beta\geq$25
PDB-trained (baseline = absolute; REPA = Δ vs. baseline)						
Baseline	0.15	0.22	0.50	0.15	0.23	0.13
REPA L9-GN	+0.01	+0.05	+0.23	−0.01	+0.01	+0.74
REPA L9-MPNN	+0.01	−0.01	+0.17	+0.11	−0.01	+0.54
REPA L4-random	*[TODO: random control — not yet β-stratified]*					
AFDB-trained						
Baseline	0.14	0.29	0.17	0.16	0.26	0.15
REPA L9-GN	+0.23	+0.02	+0.65	+0.24*	+0.06*	+0.61*
REPA L9-MPNN *(falsifier)*	+0.01	−0.02	−0.06	−0.01	−0.02	+0.00

Table 6.9: **REPA's one consistent cost — a loss of tertiary diversity — is confined to the sheet-rich folds, grows with training, and is escaped only by the AFDB-MPNN falsifier.** Concentration appears only in the $\beta\geq$25 bin, under encoders that route toward β-rich folds (GearNet on both datasets, MPNN on PDB), and grows over training (+0.23→+0.74); AFDB-MPNN-L9 stays diverse in every bin. (Designable-subset pwTM stratified by β%, lower = more diverse; Δ = REPA − baseline; red (Δ>0) = concentrates, green = more diverse, |Δ|<0.05 uncoloured. Single-seed; *900K for AFDB L9-GN.)

Variant	Quality		T-W			T-D		S-W	S-D	
	Des↑	FID↓	fJSD-A↓	fS-A↑	pwTM↓	#Clust↑	pwTM↓	ssJSD2D↓	ssJSD2D↓	E%des↑
PDB-trained, n=256, 1.0M; FID is FID-PDB.										
Baseline	0.54	467	0.95	5.67	0.17	114	0.16	0.32	0.39	0.15
REPA L9-MPNN	+0.01	−81	−0.05	+0.29	+0.07	−51	+0.15	−0.15	−0.08	+0.04
AFDB-trained, n=256, 1.0M; FID is FID-AFDB.										
Baseline	0.72	534	2.23	4.35	0.20	133	0.21	0.16	0.18	0.14
REPA L4-GN	+0.01	−228	−1.43	+3.68	−0.00	−39	+0.03	−0.12	−0.09	+0.01

Table 6.10: **The whole-set gains persist at 1.0M while designable diversity turns negative — the trade-off is temporal, not immediate.** Past the crossover the best variant per dataset keeps its FID, fS-A and fJSD-A gains but loses designable diversity (#Clust −51 PDB, −39 AFDB); contrast the 700K snapshot (Table 6.8). (n=256, step 1.0M; baseline absolute, REPA rows Δ; green / red = improvement/regression.)

β-rich folds are fold-narrow, so concentrating them costs diversity.

The trade-off is gated, temporal, and has a clean falsifier. It appears only where the encoder routes toward β-rich folds — GearNet on both datasets, MPNN on PDB — and AFDB-MPNN, which routes α-ward, escapes it entirely. It also develops with training: at 700K several variants still gain designable diversity, but by 1.0M every PDB variant has lost it while the whole-set gains persist (Table 6.10).

The two halves of the trade-off come apart cleanly. The *clustering* is a geometric inductive bias: a random-weights GearNet also reduces designable diversity, so it needs no learned features — graph-convolution over residue contacts smooths similar geometries together. The *direction* of the secondary-structure shift does need learned features; random GearNet shows no β-shift. This late, concentration-driven degradation is our

Δ (REPA − baseline)	ODE	$\gamma = 0$	$\gamma = 0.35$	$\gamma = 0.45$	$\gamma = 0.5$	$\gamma = 1.0$
PDB — REPA L9-MPNN vs Baseline (step-matched cells).						
Des ↑	+0.06	+0.07	+0.16	+0.18	+0.15	+0.04
FID ↓	−22	−1225	−78	−77	−89	−131
fJSD-A ↓	−0.13	+0.18	+0.21	−0.06	+0.07	−0.30
ssJSD2D ↓	−0.11	+0.08	−0.01	−0.03	−0.00	−0.10
#Clust ↑	+9	+8	−7	+4	+12	+2
AFDB — REPA L4-GearNet vs Baseline (step-matched cells).						
Des ↑	+0.16	+0.08	+0.07	+0.06	+0.06	+0.08
FID ↓	−102	−45	−133	−144	−132	−97
fJSD-A ↓	−0.31	+0.28	−0.94	−1.06	−0.92	−0.24
ssJSD2D ↓	−0.04	−0.19	−0.13	−0.13	−0.10	−0.03
#Clust ↑	+32	−15	−27	−23	−21	+10

Table 6.11: **REPA's gains are sampler-invariant in the middle band ($\gamma \in [0.35, 0.5]$) and largely survive at the extremes — a training-time effect that composes with the inference-time noise dial.** The large −1225 FID gain at γ=0 reflects the PDB baseline mode-collapsing at deterministic sampling. (Mean Δ = REPA − baseline across step-matched cells per noise level, for the best PDB and AFDB variants; green = improvement, red = regression; negative is a win on lower-is-better metrics.)

structural instance of *REPA works until it doesn't* [13]: as the student saturates the encoder's lower-dimensional target, alignment turns from guide to straitjacket. A schedule that switched alignment off after the early gains — which we did not test — should, on this reading, keep the whole-set gains while sparing designable diversity.

6.6 REPA's gains hold across sampler noise levels

REPA's training-time gains compose with the inference-time sampler. Proteina's SDE noise dial γ trades designability for diversity at sampling time. REPA and baseline navigate it identically — REPA shifts the *levels*, not the shape — so our γ=0.45 results are not an artefact of one sampler setting (Table 6.11).

The encoder-selectivity is itself sampler-invariant. REPA's distribution-match advantage holds at every γ for the learned encoders but is near-chance for the random control. The mechanism of §6.5 — learned structure helps, an auxiliary loss alone does not — survives the full noise sweep.

REPA also raises the deterministic-sampling floor. At γ=0 the baseline's designability collapses; REPA lifts it, durably under every learned AFDB encoder and under learned (not random) encoders on PDB (Table 6.12). The one surviving γ-failure is encoder-specific: at γ=0 GearNet-aligned REPA loses designability on both datasets, while MPNN preserves it.

ODE designability	700K	1.0M
PDB-trained	*(baseline = absolute; REPA = Δ vs. baseline)*	
Baseline	0.04	0.04
REPA L4-random	−0.02	—
REPA L4-GN	+0.16	+0.00
REPA L9-GN	+0.04	+0.09
REPA L4-MPNN	+0.12	−0.01
REPA L9-MPNN	—	+0.16
AFDB-trained		
Baseline	0.12	0.12
REPA L4-GN	+0.23	+0.22
REPA L9-GN	+0.21	+0.28*
REPA L9-MPNN	+0.10	+0.02

Table 6.12: **REPA raises the deterministic-sampling (ODE) designability floor — durably under every learned encoder on AFDB, and for learned but not random encoders on PDB.** The ODE baseline floors near-zero on PDB and modest on AFDB; REPA lifts it by +0.10 to +0.28 across steps on AFDB, while the PDB random control does not (−0.02). (ODE designability; baseline absolute, REPA rows Δ; green = higher, $|\Delta|<0.025$ uncoloured. Single-seed; *900K for AFDB L9-GN.)

6.7 What we do not claim

Several tempting claims do not survive our own data, and we flag them. REPA does not reliably improve novelty — the signal is too noisy to call. It does not "preserve β" in general; the direction is encoder×dataset conditional, and MPNN-AFDB reverses it. It does not strictly *require* a learned encoder at every scale — at n=128 a random encoder helps nearly as much (§6.8). Its designability gains are read through a proxy (ProteinMPNN→ESMFold) that shares lineage with AFDB's AlphaFold2, so we never compare designability across datasets without that caveat. We claim co-movement, not formal mediation, between representation and generation. And we retract an earlier "distribution collapse at γ=1" reading: with the full encoder grid it was a single-cell artefact.

6.8 Robustness across scale

At smaller scale the learned-versus-random gap narrows, so we anchor headline claims at n=256. At n=128 the random control helps nearly as much on representation and whole-set metrics. The direction of every gain is consistent across scale, but the margin of learned over random shrinks, so we read n=128 as a scale-robustness check rather than a headline regime. Training-dynamics ablations (λ, batch size, projector depth) are reported in Appendix A; length extrapolation — generating proteins longer than the training crop — is the natural next test and remains future work.

6.9 Summary

REPA does real mechanistic work on Proteina — but as a family of encoder-routed interventions, not a single effect. It installs structural representations the flow-matching loss never learns, as a persistent gap; it pulls the trunk's geometry onto the encoder's, generalising across encoders in the Platonic sense; and it accelerates generation, durably on synthetic data and transiently on experimental data. The representation and generation gains are encoder-matched, strong evidence that the one runs through the other. The gain is generic on whole-set distribution, encoder-specific on quality, and carries a designable-diversity trade-off that develops with training and splits into a geometric-bias clustering component and a learned-feature directional one. Which encoder you align to chooses which of these you get. We carry this — and the contrast with the small-molecule study of Chapter 5 — into Chapter 7.

Chapter 7

Conclusions

Bibliography

[1] Jonathan Ho, Ajay Jain, Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS 2020. arXiv:2006.11239.

[2] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, Matt Le. *Flow Matching for Generative Modeling*. ICLR 2023. arXiv:2210.02747.

[3] Peter Holderrieth and Ezra Erives. *An Introduction to Flow Matching and Diffusion Models*. MIT 6.S184 lecture notes, 2025. diffusion.csail.mit.edu.

[4] Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky T. Q. Chen, David Lopez-Paz, Heli Ben-Hamu, Itai Gat. *Flow Matching Guide and Code*. 2024. arXiv:2412.06264.

[5] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, Saining Xie. *Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think*. ICLR 2025. arXiv:2410.06940.

[6] Jaskirat Singh, Xingjian Leng, Zongze Wu, Liang Zheng, Richard Zhang, Eli Shechtman, Saining Xie. *What matters for Representation Alignment: Global Information or Spatial Structure?* 2025. arXiv:2512.10794.

[7] Chenyu Wang, Cai Zhou, Sharut Gupta, Zongyu Lin, Stefanie Jegelka, Stephen Bates, Tommi Jaakkola. *Learning Diffusion Models with Flexible Representation Guidance*. 2025. arXiv:2507.08980.

[8] Hyosoon Jang, Hyunjin Seo, Yunhui Jang, Seonghyun Park, Sungsoo Ahn. *Boltz is a Strong Baseline for Atom-level Representation Learning*. 2026. arXiv:2602.13249.

[9] Xiangdong Zhang, Jiaqi Liao, Shaofeng Zhang, Fanqing Meng, Xiangpeng Wan, Junchi Yan, Yu Cheng. *VideoREPA: Learning Physics for Video Generation through Relational Alignment with Foundation Models*. 2025. arXiv:2505.23656.

[10] Tri Ton, Jiajing Hong, Chang D. Yoo. *Temporally Aligned Audio for Video with Autoregression*. ICCV 2025. arXiv:2504.05684.

[11] Wei Zhang, Shijie Jin, Hangrui Zhang, Yi Li, Yan Wang. *CrystalREPA: Element-aware Representation Alignment for Crystal Generation*. 2026. arXiv:2605.08960.

[12] Runqian Wang and Kaiming He. *Diffuse and Disperse: Image Generation with Representation Regularization*. 2025. arXiv:2506.09027.

[13] Ziqiao Wang, Wangbo Zhao, Yuhao Zhou, Zekai Li, Zhiyuan Liang, Mingjia Shi, Xuanlei Zhao, Pengfei Zhou, Kaipeng Zhang, Zhangyang Wang, Kai Wang, Yang You. *REPA Works Until It Doesn't: Early-Stopped, Holistic Alignment Supercharges Diffusion Training*. 2025. arXiv:2505.16792.

[14] Tomas Geffner et al. *Proteina: Scaling Flow-based Protein Structure Generative Models*. ICLR 2025. arXiv:2503.00710.

[15] C. Vonessen et al. *TABASCO: A Fast, Simplified Model for Molecular Generation with Improved Physical Quality*. 2025. arXiv:2507.00899.

[16] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, Hongming Chen. "Molecular de-novo design through deep reinforcement learning". *Journal of Cheminformatics* 9:48 (2017). doi:10.1186/s13321-017-0235-x.

[17] Alex Zhavoronkov et al. "Deep learning enables rapid identification of potent DDR1 kinase inhibitors". *Nature Biotechnology* 37 (2019), pp. 1038–1040. doi:10.1038/s41587-019-0224-x.

[18] Joseph L. Watson et al. "De novo design of protein structure and function with RFdiffusion". *Nature* 620 (2023), pp. 1089–1100. doi:10.1038/s41586-023-06415-8.

[19] John B. Ingraham et al. "Illuminating protein space with a programmable generative model". *Nature* 623 (2023), pp. 1070–1078. doi:10.1038/s41586-023-06728-8.

[20] Amil Merchant et al. "Scaling deep learning for materials discovery". *Nature* 624 (2023), pp. 80–85. doi:10.1038/s41586-023-06735-9.

[21] Claudio Zeni et al. "A generative model for inorganic materials design". *Nature* 639 (2025), pp. 624–632. doi:10.1038/s41586-025-08628-5.

[22] Liang Wang, Chao Song, Zhiyuan Liu, Yu Rong, Qiang Liu, Shu Wu, Liang Wang. *Diffusion Models for Molecules: A Survey of Methods and Tasks*. 2025. arXiv:2502.09511.

[23] Zeming Lin et al. "Evolutionary-scale prediction of atomic-level protein structure with a language model". *Science* 379.6637 (2023), pp. 1123–1130. doi:10.1126/science.ade2574.

[24] J. Dauparas et al. "Robust deep learning–based protein sequence design using ProteinMPNN". *Science* 378.6615 (2022), pp. 49–56. doi:10.1126/science.add2187.

[25] Zuobai Zhang, Minghao Xu, Arian Jamasb, Vijil Chenthamarakshan, Aurelie Lozano, Payel Das, Jian Tang. *Protein Representation Learning by Geometric Structure Pretraining*. ICLR 2023. arXiv:2203.06125.

[26] Ilyes Batatia et al. "A foundation model for atomistic materials chemistry". *The Journal of Chemical Physics* 163.18 (2024), 184110. doi:10.1063/5.0155322.

[27] Michael Heinzinger et al. "Bilingual language model for protein sequence and structure". *NAR Genomics and Bioinformatics* 6.4 (2024), lqae150. doi:10.1093/nargab/lqae150.

[28] Jin Su et al. “SaProt: Protein Language Modeling with Structure-aware Vocabulary”. *bioRxiv* (2023). Preprint. doi:10.1101/2023.10.01.560349.

[29] Christoph Schuhmann et al. *LAION-5B: An open large-scale dataset for training next generation image-text models*. NeurIPS 2022 Datasets and Benchmarks. arXiv:2210.08402.

[30] Helen M. Berman et al. “The Protein Data Bank”. *Nucleic Acids Research* 28.1 (2000), pp. 235–242. Statistics retrieved Nov 2025, rcsb.org/stats.

[31] Mihaly Varadi et al. “AlphaFold Protein Structure Database in 2024: providing structure coverage for over 214 million protein sequences”. *Nucleic Acids Research* 52 (2024), D368–D375. doi:10.1093/nar/gkad1011.

[32] Simon Axelrod and Rafael Gómez-Bombarelli. “GEOM, energy-annotated molecular conformations for property prediction and molecular generation”. *Scientific Data* 9, 185 (2022). doi:10.1038/s41597-022-01288-4.

[33] Lorna Richardson et al. “MGnify: the microbiome sequence data analysis resource in 2023”. *Nucleic Acids Research* 51 (2023), D753–D759. doi:10.1093/nar/gkac1080. Release statistics retrieved May 2022 (2.4 billion non-redundant sequences): ebi.ac.uk.

[34] Luciano Brocchieri and Samuel Karlin. “Protein length in eukaryotic and prokaryotic proteomes”. *Nucleic Acids Research* 33.10 (2005), pp. 3390–3400. doi:10.1093/nar/gki615.

[35] Michael M. Bronstein, Joan Bruna, Taco Cohen, Petar Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. 2021. arXiv:2104.13478.

[36] Alexandre Duval, Simon V. Mathis, Chaitanya K. Joshi, Victor Schmidt, Santiago Miret, Fragkiskos D. Malliaros, Taco Cohen, Pietro Liò, Yoshua Bengio, Michael Bronstein. *A Hitchhiker’s Guide to Geometric GNNs for 3D Atomic Systems*. 2024. arXiv:2312.07511.

[37] Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, Patrick Riley. *Tensor Field Networks: Rotation- and Translation-Equivariant Neural Networks for 3D Point Clouds*. 2018. arXiv:1802.08219.

[38] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. arXiv:2010.11929.

[39] Jinwoo Kim, Tien Dat Nguyen, Seonwoo Min, Sungjun Cho, Moontae Lee, Honglak Lee, Seunghoon Hong. *Pure Transformers are Powerful Graph Learners*. NeurIPS 2022. arXiv:2207.02505.

[40] John Jumper et al. “Highly accurate protein structure prediction with AlphaFold”. *Nature* 596 (2021), pp. 583–589. doi:10.1038/s41586-021-03819-2.

[41] Josh Abramson et al. “Accurate structure prediction of biomolecular interactions with AlphaFold 3”. *Nature* 630 (2024), pp. 493–500. doi:10.1038/s41586-024-07487-w.

[42] Johann Brehmer et al. *Does equivariance matter at scale?* NeurIPS 2024. arXiv:2410.23179.

[43] Jason Yim et al. *Fast protein backbone generation with SE(3) flow matching*. 2023. arXiv:2310.05297.

[44] Yeqing Lin and Mohammed AlQuraishi. *Generating Novel, Designable, and Diverse Protein Structures by Equivariantly Diffusing Oriented Residue Clouds*. ICML 2023. arXiv:2301.12485.

[45] Tuan Le et al. *Navigating the design space of equivariant diffusion-based generative models for de novo 3D molecule generation*. ICLR 2024. arXiv:2309.17296.

[46] Clement Vignac et al. *MiDi: Mixed Graph and 3D Denoising Diffusion for Molecule Generation*. ECML PKDD 2023. arXiv:2302.09048.

[47] Ross Irwin et al. *Efficient 3D Molecular Generation with Flow Matching and Scale Optimal Transport*. 2024. arXiv:2406.07266.

[48] National Institutes of Health. *Regulatory Knowledge Guide for Small Molecules*. NIH SEED, 2024. seed.nih.gov.

[49] Encyclopaedia Britannica. *What is the difference between a peptide and a protein?* britannica.com.

[50] Gisbert Schneider. "Automating drug discovery". *Nature Reviews Drug Discovery* 17 (2018), pp. 97–113. doi:10.1038/nrd.2017.232.

[51] Martin Buttenschoen, Garrett M. Morris, and Charlotte M. Deane. "PoseBusters: AI-based docking methods fail to generate physically valid poses or generalise to novel sequences". *Chemical Science* 15 (2024), pp. 3130–3139. arXiv:2308.05777.

[52] G. Richard Bickerton et al. "Quantifying the chemical beauty of drugs". *Nature Chemistry* 4 (2012), pp. 90–98. doi:10.1038/nchem.1243.

[53] Pascal Notin et al. "Machine learning for functional protein design". *Nature Biotechnology* 42 (2024), pp. 216–228. doi:10.1038/s41587-024-02127-0.

[54] Yeqing Lin, Minji Lee, Zhao Zhang, and Mohammed AlQuraishi. "Out of Many, One: Designing and Scaffolding Proteins at the Scale of the Structural Universe with Genie 2". *arXiv preprint* arXiv:2405.15489 (2024). arXiv:2405.15489.

[55] Christine A. Orengo, A. D. Michie, S. Jones, David T. Jones, M. B. Swindells, Janet M. Thornton. "CATH — a hierarchic classification of protein domain structures". *Structure* 5.8 (1997), pp. 1093–1108. doi:10.1016/S0969-2126(97)00260-8.

[56] Minyoung Huh, Brian Cheung, Tongzhou Wang, Phillip Isola. *The Platonic Representation Hypothesis*. ICML 2024. arXiv:2405.07987.

Appendix A

Technical details

A.1 Placeholder section

TODO.